React

DAWeb

Grado en Informática

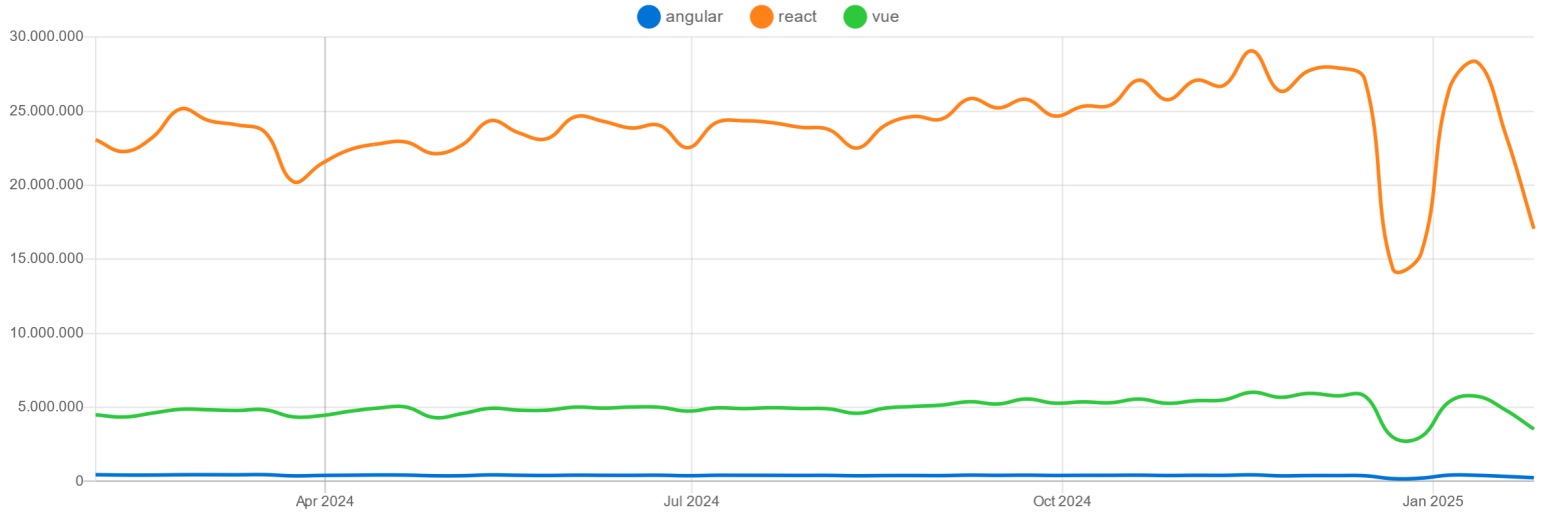
Curso 2025/26

React

- React es una librería de JavaScript declarativa y flexible para construir interfaces de usuario. https://react.dev
- Es una librería open source de JavaScript para desarrollar interfaces de usuario.
- Permite componer IUs formadas por piezas de código pequeñas y modulares, llamadas "componentes".

React

- Lanzada en el año 2013 y desarrollada por Facebook, quienes también la mantienen actualmente junto a una comunidad de desarrolladores independientes y compañías.
  - Facebook, Instagram, Netflix, AirBnB,...



React

- Como norma general, al diseñar una aplicación con React, lo que estamos haciendo es crear componentes independientes y reusables.
- Toda aplicación en React tiene al menos un componente: componente raíz (App habitualmente).
- Finalmente la página es un árbol de componentes.
- La comunicación es unidireccional de padres a hijos en el árbol de componentes.
- A partir de la versión 16.8 de la librería de React la propuesta por defecto es el empleo de componentes funcionales en lugar de clases.

React y JSX

- JSX es el lenguaje para definir código javascript con sintaxis XML. JSX es más estricto que HTML.
- Tienes que cerrar todas las etiquetas como: <br/>.
- Tu componente tampoco puede devolver múltiples etiquetas de JSX. Debes envolverlas en un padre compartido, como ... o en un envoltorio vacío <>...</>.
- Formato para ser compilado y generar Javascript puro.

React y JSX

- Contenido de las etiquetas y valores de los atributos pueden ser evaluados como expresiones dentro de '{}'.
- Objetos anidados dentro de '{{}}':
  - Ejemplo:'{{nombre:"Jose", tlf:868888556}}'.
- Atributo HTML **class** se cambia por **className**.
- Atributo HTML **for** se cambia por **htmlFor**.

React y JSX

- Dentro del bloque JSX podemos acceder a variables o constantes indicando entre llaves dicha variable o constante.

```
<p>Hola {nombre}</p>
```

- Podemos usar expresiones para acceder a propiedades de un objeto previamente definido:

```
<p>{persona.nombre} tiene {persona.edad} años</p>
```

React y JSX

- Otra posibilidad en una expresión es hacer la llamada a otras funciones:

```
{retornarAleatorio()}
```

- Inclusive disponer una operación que será ejecutada previo a su visualización:
  - 3 + 3 = {3 + 3}

React y JSX

- Uso de convertidores como transform.tools o babeljs para traducir código HTML a JSX.

```
<!-- Código HTML -->
<label for="nombre" style="font-size:1.5em;">Nombre: </label>
<input type="text" id="nombre" />
```

```
<>
  {/* Código JSX */}
  <label htmlFor="nombre" style={{ fontSize: "1.5em" }}>
    Nombre:{" "}
  </label>
  <input type="text" id="nombre" />
</>
```

Virtual DOM

- La visualización devuelta genera la vista del componente utilizando el lenguaje JSX, y permite acceder a la lógica.

- A través de este código JSX se crea una representación en memoria del DOM ("Virtual DOM"), a partir de la cual luego crea el DOM definitivo en el navegador.

Virtual DOM

- "Virtual DOM", a diferencia del DOM común, pesa muy poco y utiliza escasos recursos para ser creado.
- Cuando un componente cambia su estado (por tanto la información a renderizar/mostrar), React comprueba los cambios realizados en Virtual DOM del componente y actualiza el árbol DOM de la página solo las partes de Virtual DOM que ha cambiado.
- Al tratarse de código JavaScript, la compilación y el renderizado de los cambios se realiza rápidamente.

React DOM

- Paquete responsable de interactuar con el DOM que se renderiza en el navegador con React. Se encarga de renderizar componentes React y gestionar interacciones con el DOM.

```
import ReactDOM from "react-dom/client";
import App from "./App";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

Vite

- Es una herramienta de compilación que tiene como objetivo proporcionar una experiencia de desarrollo más rápida y ágil para proyectos web modernos. Web Vite
- Proporciona:
  - Un servidor de desarrollo con Hot Module Replacement (HMR).
  - Un comando de compilación que empaqueta el código con Rollup.
  - Es compatible con React, pero también con Vue, Svelte, Preact, ...

Vite - Inicialización

- Crear proyecto:

```
npm create vite@latest <nombreProyecto> --template -react
```

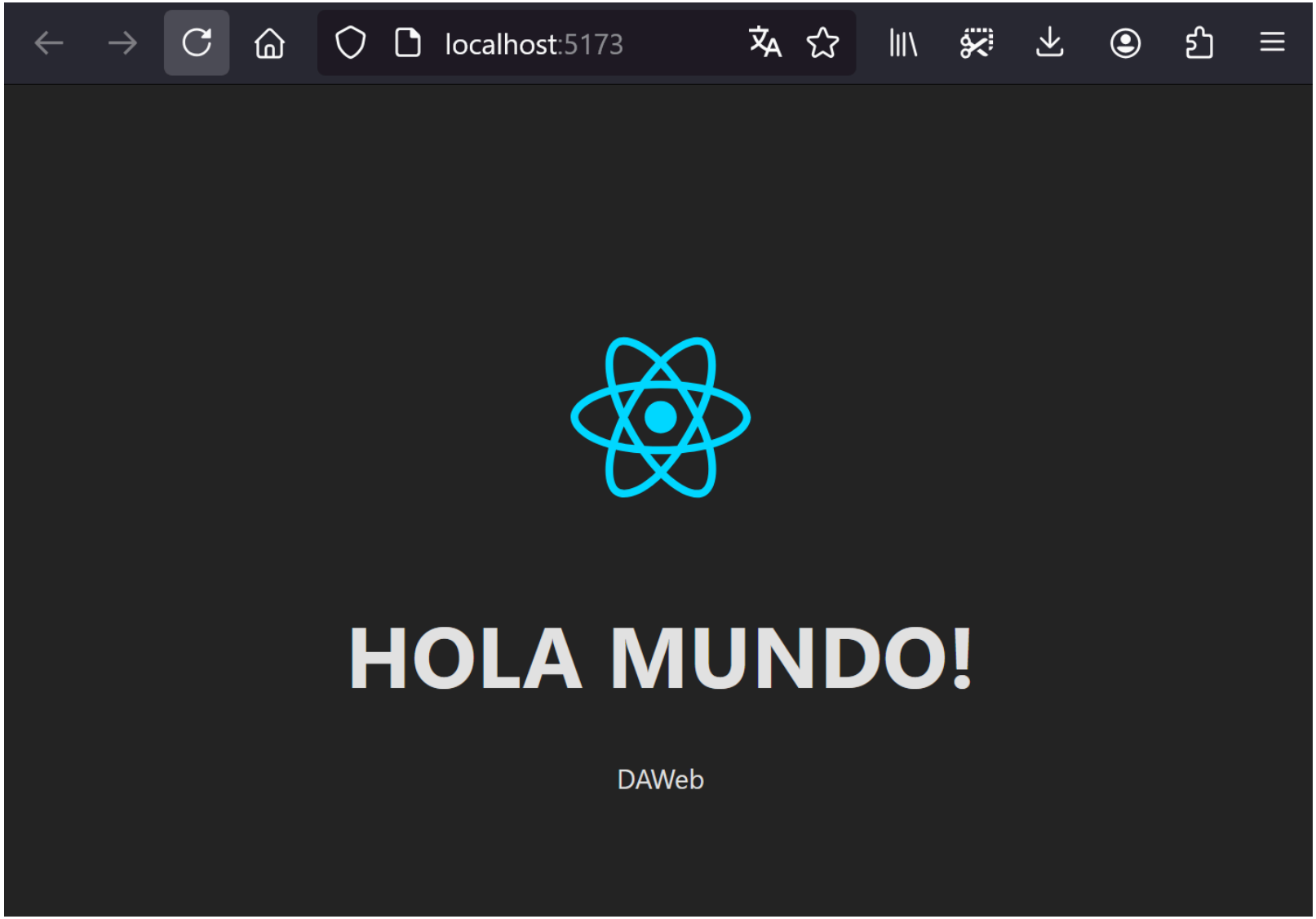
- Instalar dependencias:

```
npm install
```

- Iniciar servidor:

```
npm run dev
```

Proyecto



Proyecto

- Ejemplo de **index.html**:

```html
<!doctype html>
<html lang="en">
  <head>
    <script type="module" src="/@vite/client"></script>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Proyecto - index.html

- Contiene un elemento div donde de forma dinámica y mediante Javascript se agregaran en su interior los elementos HTML de nuestro programa.
  - Incluye el script asociado con el modulo **main.jsx** que se encarga de la lógica con los componentes de la página.

Proyecto - main.jsx

- Sigue módulos ES6, los archivos terminan en **.jsx**.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Proyecto - App.jsx

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import ConcertInfo from './component/component'
import './App.css'

function App() {
  const [count, setCount] = useState(0)
  return (
    <>
      <div>
        <a href="https://vite.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}> count is {count} </button>
        <p>Edit <code>src/App.jsx</code> and save to test HMR</p>
        <ConcertInfo
          concert={{name: 'Music Festival',
            date: '2022-06-01', venue: 'Central Park',
            city: 'New York', price: 100,}}/>
      </div>
      <p className="read-the-docs">Click on the logos to learn more</p>
    </>
  )
}
export default App;
```

Componentes

- Son los elementos básicos que se reutilizan para formar los interfaces de los usuarios.

- Se implementan como funciones javascript.
  - **NOTA**: el nombre de las funciones deben empezar por mayúscula.
  - Se definen con etiquetas html usando la sintaxis JSX u otros componentes.

Componentes

- No deberían definirse componentes dentro de la declaración de la función de otro componente.
- Para pasar información de un componente padre a otro hijo se utilizan propiedades u objetos.
- Se pueden pasar datos de tipo cadena, numérico, booleano, objeto, array.
- También puede recibir código JSX u otro componente React.

Componentes

```
const Mensaje = ({titulo, numero, booleano, params}) =>
    (<div>
        <table>
            <thead><tr><th>{titulo}</th></tr></thead>
            <tbody>{params.msg.map((m,index) =>
                (<tr><td>{m}</td></tr>))}
            </tbody>
            <tfoot>
                <tr><th>{numero}</th></tr>
                <tr><th>{booleano ? "Verdadero" : "Falso"}</th></tr>
            </tfoot>
        </table>
    </div>);
```

```
<Mensaje titulo="Mensajes" numero={2025}
  booleano={false} params={{msg:["msg1", "msg2"]}}/>
```

Componentes

- Los elementos en una lista deben tener una clave propia para facilitar su renderizado en React.
- Se utiliza la propiedad **key** para indicar ese identificador único entre los elementos de la lista.

```
const Mensaje = ({titulo, numero, booleano, params}) =>
  (<div>
    <table>
      <thead><tr><th>{titulo}</th></tr></thead>
      <tbody>{params.msg.map((m,index) =>
        (<tr key={index}><td>{m}</td></tr>))}
      </tbody>
      <tfoot>
        <tr><th>{numero}</th></tr>
        <tr><th>{booleano ? "Verdadero" : "Falso"}</th></tr>
      </tfoot>
    </table>
  </div>);
```

Componentes

- Ejemplo de paso de código JSX.

```
const Titulo = ({titulo}) => (<h2>{titulo}</h2>)

const Mensaje=({titulo}) => (
  <div>
    <Titulo titulo={titulo}/>
  </div>
)
```

Componentes

- Ejemplo de paso de componente React.

```
const Mensaje=({titulo}) => (
  <div>
    {titulo}
  </div>
)
```

```
const Titulo = ({titulo}) => (<h2>{titulo}</h2>)
    <Mensaje titulo={<Titulo titulo="Mensajes"/>}/>
```

Eventos

- Los nombres de eventos en React comienzan con **on** : onClick, onDoubleClick, onMouseEnter, onMouseLeave, onMouseMove, onKeyPress, onKetUp, onSubmit, etc.

```
function App() {
return (
<div>
  <form onSubmit={enviar}>
    <p>Ingrese primer valor:<input type="text" name="valor1" /></p>
    <p>Ingrese segundo valor:<input type="text" name="valor2" /></p>
    <input type="submit" value="Sumar" />
  </form>
</div>
)
```

Eventos

```
function enviar(e) { // e : Event, e.target : generador del Event
  e.preventDefault();
  const v1 = parseInt(e.target.valor1.value);
  const v2 = parseInt(e.target.valor2.value);
  alert('La suma es:'+ (v1+v2));
}
export default App;
```

- **Recordatorio**: En la función enviar primero se detiene el envío de datos al servidor llamando a *preventDefault()* para evitar la acción por defecto.

Hooks

- Nos permiten gestionar diferentes características de los componentes basados en funciones.

- Hook de *estado*, almacenan información del componente: **useState**, **useReducer**.

- Hook de *efecto*, permiten la sincronización con recursos externos: **useEffect**.

Hooks

- Hook de *contexto*, paso de información de un componente padre a otro independientemente del nivel de anidamiento en el que esté: **useContext**.

- Hook de *referencia*, permite definir una referencia a un elemento o dato que no implique un cambio en el renderizado de un componente: **useRef**.

- Hook de *rendimiento*, permiten evitar renderizar un componente cuyos valores en el hook no cambien: **useCallback**, **useMemo**.

Hooks - useState

- El valor del componente se modifica mediante el método **set** asociado.
- El valor inicial se indica en la declaración del hook de estado.
- El hook de estado indica con un array con la variable de la información que se almacena como primer elemento, y el método de actualización en el segundo elemento.
    - La variable puede se de cualquier tipo: cadena, número, array, booleano,...

Hooks - useState

```
function ContadorState({valorInicial}){
    const [contador, setContador] = useState(valorInicial);
    return (
        <div>
            <h2>{contador}</h2>
            <button onClick={()=>setContador(contador+1)}>
              Incrementar
            </button>
            <button onClick={()=>setContador(contador-1)}>
              Decrementar
            </button>
        </div>
    );
}
```

Hooks - useReducer

- El componente tiene un estado más complejo y se le asocia una lógica.
- Parecido al hook *useState*.
- Se cambia el método *set* para actualizar el valor almacenado por un método *dispatch*.
- Declara una función *reducer* que implementa una lógica más compleja de modificación del estado del valor almacenado.

Hooks - useReducer

```
function reducer(estado, accion) {
    switch (accion.tipo) {
        case 'incrementar':
            return { contador: estado.contador + 1 };
        case 'decrementar':
            return { contador: estado.contador - 1 };
    }
}

function ContadorReducer({contadorInicial}) {
    const estadoInicial = { contador: contadorInicial};
    const [estado, enviar] = useReducer(reducer, estadoInicial);

    return (
        <div>
            <h2>{estado.contador}</h2>
            <button onClick={() => enviar({ tipo: 'incrementar' })}>Incrementar</button>
            <button onClick={() => enviar({ tipo: 'decrementar' })}>Decrementar</button>
        </div>
    );
}
```

Hooks - useEffect

- Utilizado normalmente para implementar funcionalidad secundaria como:
  - Accedder manualmente al DOM.
  - Invocar a APIs mediante fetch o Axios.
  - Manejar eventos (*addEventListener*,*removeEventListener*).

Hooks - useEffect

```
const ContadorEffect = () => {
  const [contador, setContador] = useState(() => {
    const valorInicial = localStorage.getItem("contador");
    return valorInicial !== null ? parseInt(valorInicial) : 0; }
  );
  const [autoIncremento, setAutoIncremento] = useState(false);

  useEffect(() => {
    localStorage.setItem("contador", contador);
  }, [contador]);

  useEffect(() => {
    if (autoIncremento) {
      const interval = setInterval(() => {
        setContador((contadorAnterior) => contadorAnterior + 1);
      }, 1000);
      return () => clearInterval(interval);
    }
  }, [autoIncremento]);

  return (
    <div>
      <h2>Contador: {contador}</h2>
      <button onClick={() => setAutoIncremento(!autoIncremento)}>
        {autoIncremento ? "Detener" : "Iniciar"}
      </button>
    </div>
  );
};
```

Hooks - useContext

- Define un contexto que cualquier componente pueda acceder de manera global.

```
import React, { createContext, useContext, useState } from "react";
const ContadorContext = createContext();

export const ContadorProveedor = ({ children }) => {
  const [contador, setContador] = useState(0);
  const incrementar = () => {
    setContador(contador + 1);
  };
  const decrementar = () => {
    setContador(contador - 1);
  }
  return (<ContadorContext.Provider value={{ contador, incrementar, decrementar }}>
            {children}
          </ContadorContext.Provider> );
};
export const useContador = () => useContext(ContadorContext);
```

Hooks - useContext

- Ejemplo de uso de un contexto en un componente.

```
import React, { useContext } from 'react';
import { useContador } from '../component/contadorContext';

const ContadorConContexto = () => {
    const { contador, incrementar, decrementar } = useContador();
    return (
        <div>
            <h1>Contador: {contador}</h1>
            <button onClick={incrementar}>Incrementar</button>
            <button onClick={decrementar}>Decrementar</button>
        </div>
    );
};
```

Hooks - useRef

- Se utiliza para actualizar información del componente que no tenga una acción directa sobre su renderizado.
    - Al usar *useRef* se pasa como argumento el valor inicial de la referencia.
    - Utiliza la propiedad **current** que contiene el valor de la referencia en el componente.

Hooks - useRef

```
const ContadorRef = () => {
  const [contador, setContador] = useState(0);
  const intervaloRef = useRef(null);
  const iniciar = () => {
    if (!intervaloRef.current) {
      intervaloRef.current = setInterval(() => {
        setContador((contador) => contador + 1);
      }, 1000);
    }
  };
  const detener = () => {
    clearInterval(intervaloRef.current);
    intervaloRef.current = null;
  };
  return (
    <div>
      <h1>ContadorRef: {contador}</h1>
      <button onClick={iniciar}>Iniciar</button>
      <button onClick={detener}>Detener</button>
    </div>
  );
};
```

Hooks - useMemo

- Sirve para evitar que una función se ejecute cada vez que se renderiza el componente si los valores no cambian.

- Almacena en una *cache* el resultado de la ejecución de la función a memorizar.

- Recibe dos argumentos: la función a ejecutar y las dependencias para volver a ejecutar la función.

Hooks - useMemo

```
function ContadorPrimos() {
  const [limite, setLimite] = useState(100);
  const calcularPrimos = (limite) => {
    let primos = [];
    for (let i = 2; i <= limite; i++) {
      if (primos.every((primo) => i % primo !== 0)) {
        primos.push(i);
      }
    }
    return primos;
  };

  const numerosPrimos = useMemo(() => calcularPrimos(limite), [limite]);

  return (
    <div>
      <h2>Numeros Primos hasta {limite}</h2>
      <p>Total: {numerosPrimos.length}</p>
      <button onClick={() => setLimite(limite + 100)}>Aumentar límite ({limite})</button>
    </div>
  );
}
```

Comunicación entre componentes

1. Paso de información de un componente nivel superior o padre a un componente anidado o hijo mediante propiedades.
2. Paso de información de un componente anidado a uno de nivel superior, mediante el paso de funciones que modifiquen su estado.
3. Paso de información entre componentes no directamente relacionados usando el hook **useContext**.

Comunicación entre componentes

- Ejemplo de paso de funciones entre componente padre e hijo.

```
function ContadorPadre() {
    const [contador, setContador] = useState(0);
    const incrementar = () => {
        setContador(contador + 1);
    };
    const decrementar = () => {
        setContador(contador - 1);
    };
    return (
        <div>
          <h1>ContadorPadre: {contador}</h1>
          <BotonContador funcion={incrementar} texto="Incrementar" />
          <BotonContador funcion={decrementar} texto="Decrementar" />
        </div>
    );
}
```

Comunicación entre componentes

```
import React from "react";

const BotonContador = ({ funcion, texto }) => {
    return (
        <button onClick={funcion}>{texto}</button>
    );
};

export default BotonContador;
```

Componente - ciclo de vida

- El ciclo de vida de un componente se divide en montaje, actualización y desmontaje.
  - Se utiliza el Hook **useEffect** para manejar dicha funcionalidad.
  - *Montaje* se invoca la primera vez que se añade el componente en el DOM.
  - *Actualización* se ejecuta cada vez que se modifica la vista asociada al componente en el DOM.
  - *Desmontaje* se invoca cuando se quita el componente del árbol DOM.

Componente - ciclo de vida

```
const Componente = () => {
  const [contador, setContador] = useState(0);

  useEffect(() => {
    console.log("Componente montado");
    return () => {
      console.log("Componente desmontado");
    };
  }, []); //No se añade ningún elemento, se ejecuta solo una vez.

  useEffect(() => {
    console.log("Componente acutalizado");
  }, [contador]); //Se ejecuta cada vez que se cambia el valor del contador

  return (
    <div>
      <h1>Contador: {contador}</h1>
      <button onClick={() => setContador(contador + 1)}>Incrementar</button>
    </div>
  );
};
```

Enrutamiento

- Para manejar varias rutas de la aplicación se usa **react-router-dom**. Extiende al paquete **react-router**.

- *BrowserRouter*: habilita el enrutamiento en la página.

- *Routes*: contiene todas las rutas de la aplicación y se encarga de evaluar la primera que coincida con la URL.

Enrutamiento

- *Route*: define las rutas individuales con la propiedad **path**, y con la propiedad **element** se especifica el componente que debe renderizarse o código JSX.

- *Link*: es utilizado para crear enlaces de navegación dentro de una aplicación React sin que se realice una recarga completa de la página. Se utiliza el atributo **to** para indicar la ruta a la que ir.

Enrutamiento

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import App from "./App";

const Rutas = () => {
  return (
    <BrowserRouter>
      <nav><ul>
        <li><Link to="/">Home</Link></li>
        <li><Link to="/about">About</Link></li>
      </ul></nav>
      <Routes>
        <Route path="/" element={<App />} />
        <Route path="/about" element={<h1>About</h1>} />
        <Route path="*" element={<h1>Not Found</h1>} />
      </Routes>
    </BrowserRouter>
  );
};
```

Componentes predefinidos

- Exiten muchos componentes React ya implementados y de código abierto que se pueden reutilizar.
- react-bootstrap

```
npm install react-bootstrap bootstrap
```

- Material-UI

```
npm install @mui/material @emotion/react @emotion/styled
```

- react-toastify

```
npm i --save react-toastify
```

Componentes predefinidos

- Ejemplo con **react-toastify**:

```
import { ToastContainer, toast } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

function App() {
  const notificar = () => toast("¡Hola Mundo!");

  return (
    <div>
      <button onClick={notificar}>Notificar</button>
      <ToastContainer />
    </div>
  );
}
```

Fin

Anexo - ES6 Module

- Módulo con sufijo **.jsx**

```
/*Fichero modulo.jsx*/
export function suma(a,b){
  return a+b;
}

export default resta(a,b){
  return a-b;
}
```

Anexo - ES6 Module

- Importar functiones de módulos, incluyendo funciones por defecto.

```
import resta, {suma} from 'modulo.jsx';

resta(5, 2);
suma(1,2);
```